

# Optional: Advanced Migrations

Change an existing schema deliberately.

Use the separate [ORM reference project](#), built with `run2-optional.sh`. It includes all migrations below. It uses its own practice database, so its schema differs from lesson 4's `students` table.

# Learning Goals

- Change an existing schema with a new migration, such as adding and removing a column.
- Choose column rules, foreign keys, and indexes.
- Compare rollback, reset, and fresh, know which ones lose data, and seed repeatably.

## Add a New Migration

```
php artisan make:migration add_gpa_to_students_table --table=students
```

In a project you build yourself, use a new migration for an applied schema.

In the reference app, inspect the existing GPA migration instead. Editing an old migration does not change a database that already ran it.

# Add and Remove a Column

```
public function up(): void
{
    Schema::table('students', function (Blueprint $table) {
        $table->decimal('gpa', 3, 2)->nullable();
    });
}
public function down(): void
{
    Schema::table('students', function (Blueprint $table) {
        $table->dropColumn('gpa');
    });
}
```

Rolling back removes stored GPA values.

# Verify a GPA Rollback

The reference setup applies GPA last, in its own batch.

Confirm this with `migrate:status` before rolling back:

```
php artisan migrate:status  
php artisan migrate:rollback --step=1
```

In a new Tinker session:

```
Schema::hasColumn('students', 'gpa'); // false  
App\Models\Student::count();          // students remain
```

Run `php artisan migrate --step` to re-add GPA with null values.

Rolling back removes GPA values but keeps the student records.

## Choose Column Rules

```
$table->string('email')->unique();  
$table->decimal('gpa', 3, 2)->nullable();  
$table->boolean('is_active')->default(true);
```

- `unique()` rejects duplicates at the database level.
- `nullable()` allows a missing value.
- A default supplies a value when an insert omits the field.

Request validation should agree with database constraints.

## Link Two Tables

Create `authors` before `books` . In the books migration:

```
Schema::create('books', function (Blueprint $table) {  
    $table->id();  
    $table->string('title');  
    $table->foreignId('author_id')->constrained();  
    $table->timestamps();  
});
```

This references `authors.id` . Choose deletion behavior explicitly  
when your project needs cascading deletes or another policy.

## Add an Index for a Query

For an existing students table with `major` and `year` :

```
Schema::table('students', function (Blueprint $table) {  
    $table->index(['major', 'year']);  
});
```

In `down()` , use `$table->dropIndex(['major', 'year']);`  
inside `Schema::table`.

Indexes can speed reads but add storage and write cost.



# Rollback, Reset, and Fresh

| Command                                | Effect                          |
|----------------------------------------|---------------------------------|
| <code>migrate:rollback</code>          | Reverse the latest batch        |
| <code>migrate:rollback --step=2</code> | Reverse the last two migrations |
| <code>migrate:reset</code>             | Reverse all applied migrations  |
| <code>migrate:fresh</code>             | Drop all tables, then migrate   |

Inspect `down()` and the database target first. Data may be lost.

# Seed a Repeatable Example

The reference includes `OrmPracticeSeeder`. Its student setup uses:

```
Student::updateOrCreate(  
    ['email' => 'practice@example.com'],  
    ['name' => 'Practice Student', 'major' => 'CS', 'year' => 2, 'is_active' => true]  
);
```

Run `php artisan db:seed --class=OrmPracticeSeeder`.  
Repeating this seed updates the same email instead of duplicating it.

## Practice a Schema Change

1. Save a student in a dedicated practice database.
2. Apply the GPA migration; verify the original student remains.
3. Store a GPA, verify it is the latest migration, then roll back one step.
4. Confirm the student remains but the GPA column is gone.

Use `migrate:status` before and after. Do not use `fresh` for this exercise.